01 Vectors

1.3 Vector Addition and Subtraction

From the Parallelogram Law to the Triangle Law

Linear Algebra Made Easy and Visual | Open-source code & drafts: https://github.com/Visualize-ML

SECTION OVERVIEW

What You Will Learn

1. Understand the algebraic definition: adding/subtracting n-D vectors component by component
2. Grasp the geometry of vector addition: the parallelogram law and the triangle law
3. Understand and verify the commutative law $\boldsymbol{a}+\boldsymbol{b}=\boldsymbol{b}+\boldsymbol{a}$ and the associative law
4. Master the parallelepiped rule: adding three (non-coplanar) vectors in space
5. Understand the zero vector, the opposite vector, and $a-b=a+(-b)$
6. Implement vector addition/subtraction and its visualization with NumPy and Matplotlib

SECTION FLOW

How the Ideas Connect

1. Addition: component by component
2. RGB two-/three-color mixing
3. The parallelogram law
4. Zero vector & collinear case
5. Parallelepiped: three vectors
6. Triangle law: tip-to-tail
7. Commutative & associative laws
8. Subtraction: $\boldsymbol{a-b}$

ALGEBRAIC DEFINITION

Vector Addition: Component by Component

$$\boldsymbol{a}+\boldsymbol{b}=\begin{bmatrix}a_1\\a_2\\\vdots\\a_n\end{bmatrix}+\begin{bmatrix}b_1\\b_2\\\vdots\\b_n\end{bmatrix}=\begin{bmatrix}a_1+b_1\\a_2+b_2\\\vdots\\a_n+b_n\end{bmatrix}$$

- Two n-D vectors ***a*** and ***b*** must share the same dimension to be added
- Each pair of corresponding components is added; the result is again an n-D vector

- $a_1, a_2, \ldots, a_n$ are the scalar components of ***a*** (unknowns shown in italic)
- Next, a one-line NumPy check: a_vec + b_vec

CODE WALKTHROUGH · LA_01_03_01.IPYNB

Vector Addition with NumPy

np.array() builds the vectors, + adds them component-wise

- Import NumPy, create vectors a_vec and b_vec with np.array()
- Run a_vec + b_vec — NumPy adds them component by component
- No explicit loop is needed; vectorized code stays concise and fast
- The result is again a 1-D NumPy array, i.e. a new vector

```python
## 初始化
import numpy as np

## 定义向量
a_vec = np.array([4, 1])
b_vec = np.array([1, 2])

## 向量加法
a_plus_b = a_vec + b_vec
```

Figure 1 · Adding Two Primary Colors

- $\boldsymbol{e_1}$=pure red, $\boldsymbol{e_2}$=pure green, $\boldsymbol{e_3}$=pure blue; pairwise sums give secondary colors
- $\boldsymbol{e_1}+\boldsymbol{e_2}$=yellow, $\boldsymbol{e_1}+\boldsymbol{e_3}$=magenta, $\boldsymbol{e_2}+\boldsymbol{e_3}$=cyan — mixing light is vector addition

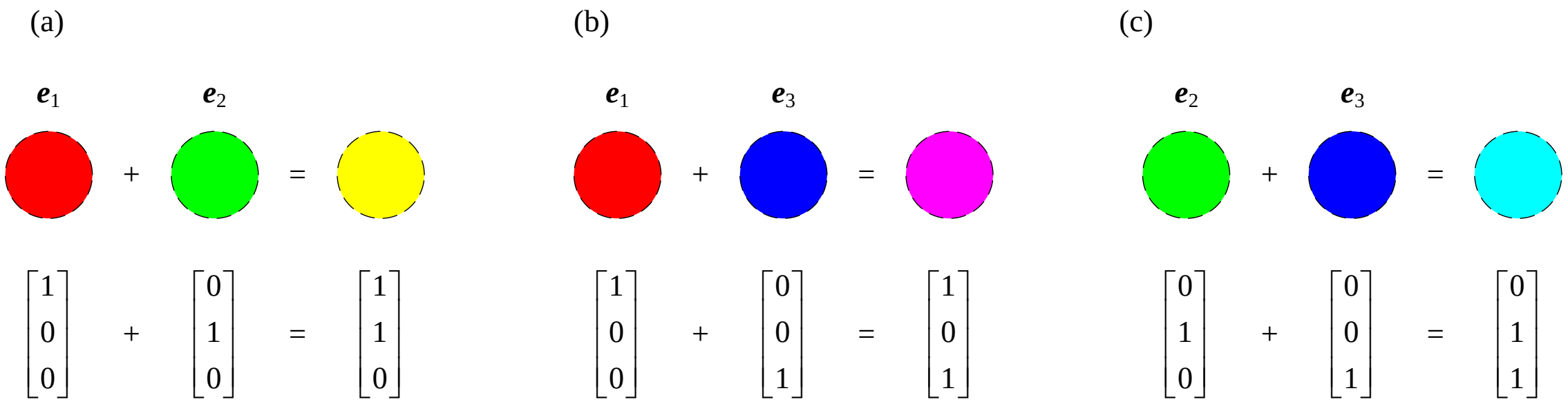
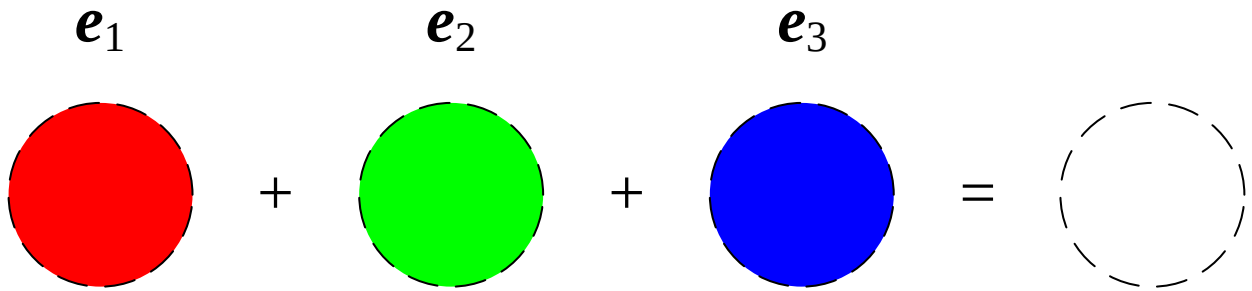


Figure 2 · Adding Three Primary Colors

- Adding all three base-color vectors: $\boldsymbol{e_1}+\boldsymbol{e_2}+\boldsymbol{e_3}$ = white
- A natural extension of addition from two vectors to three
- Still component-wise: $[1,0,0]^T+[0,1,0]^T+[0,0,1]^T=[1,1,1]^T$
- $[1,1,1]^T$ is exactly the vertex of the RGB cube that represents pure white



$$\begin{bmatrix}1\\0\\0\end{bmatrix} + \begin{bmatrix}0\\1\\0\end{bmatrix} + \begin{bmatrix}0\\0\\1\end{bmatrix} = \begin{bmatrix}1\\1\\1\end{bmatrix}$$

GEOMETRIC VIEW

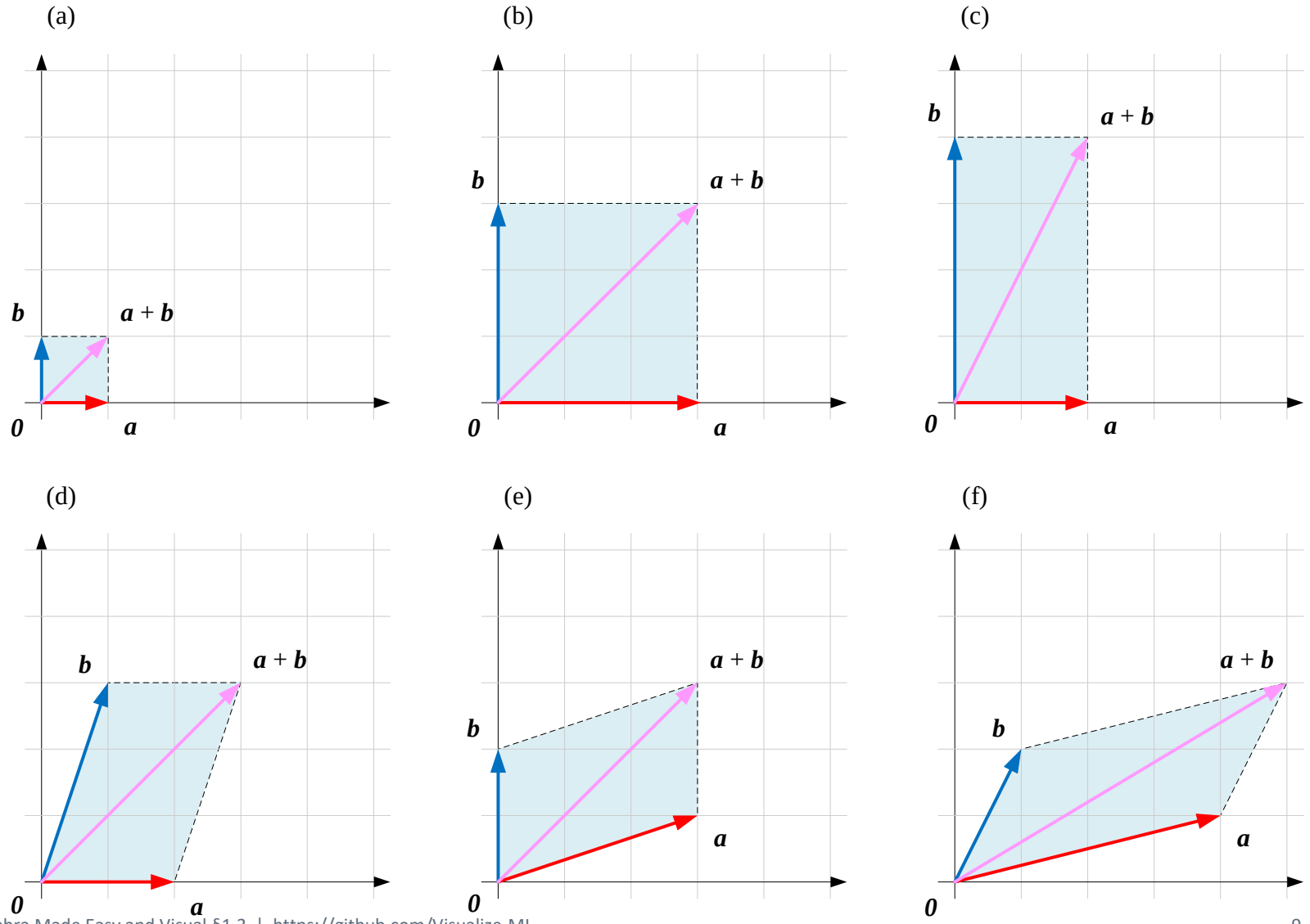
The Parallelogram Law

$\boldsymbol{a}$, $\boldsymbol{b}$ share a start point → complete a parallelogram with $\boldsymbol{a}$, $\boldsymbol{b}$ as sides → the diagonal is $\boldsymbol{a}+\boldsymbol{b}$

- Translate $\boldsymbol{a}$ and $\boldsymbol{b}$ to a common starting point, then complete a parallelogram using them as adjacent sides
- The diagonal leaving the common start point has the direction and length of $\boldsymbol{a}+\boldsymbol{b}$
- This is the classic geometric picture of vector addition — think resultant force or resultant velocity

Figure 3 · Examples of the Parallelogram Law

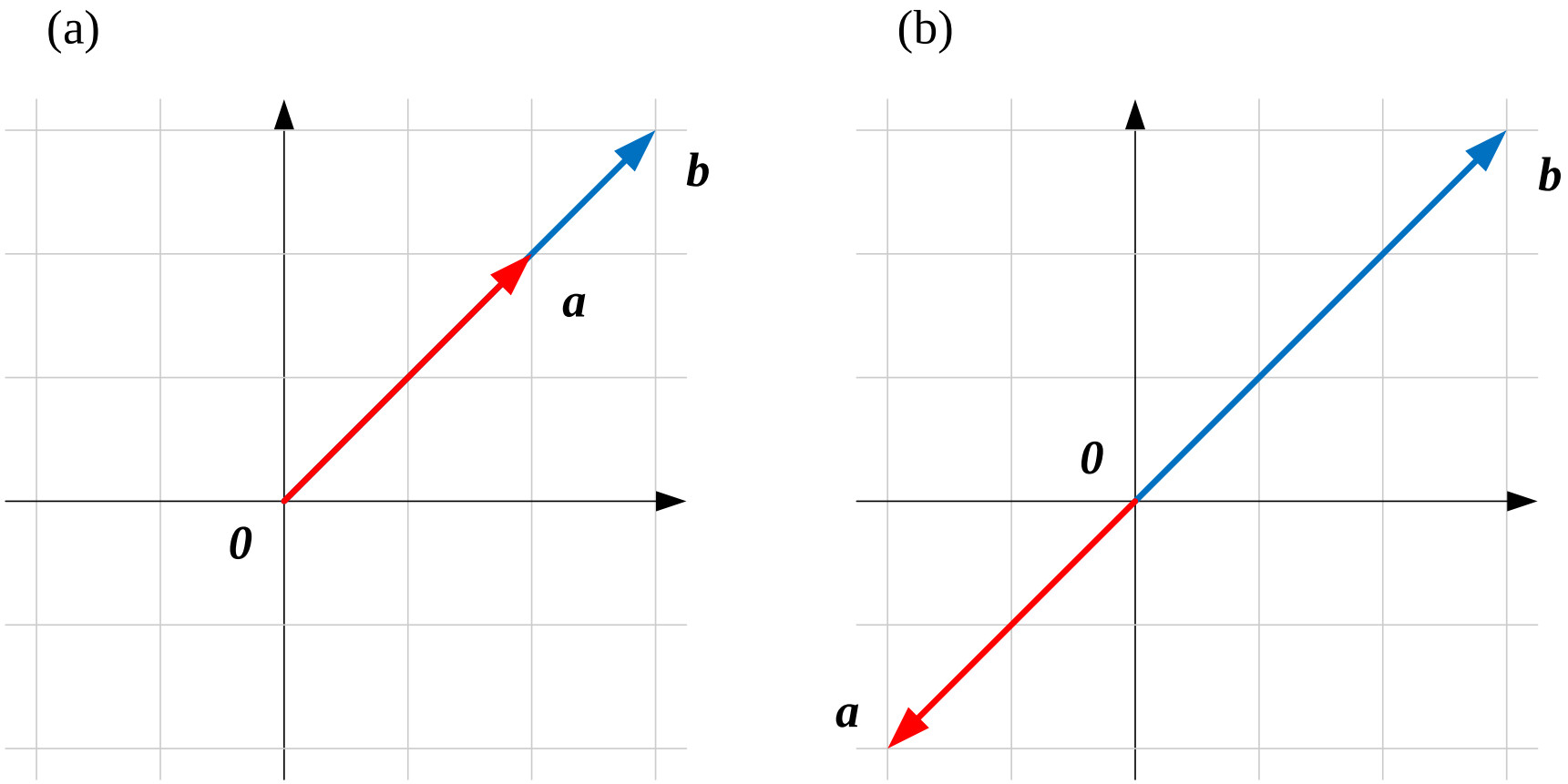
- (a)–(c) unit square, square, rectangle: ***a*** and ***b*** are perpendicular
- (d)–(f) general parallelograms: ***a*** and ***b*** meet at an arbitrary angle — the law still holds



SPECIAL CASE

Figure 4 · a and b Are Collinear

- When $\boldsymbol{a}$ and $\boldsymbol{b}$ point the same or opposite way (collinear), no true parallelogram can be formed
- The "parallelogram" degenerates to a line segment; $\boldsymbol{a}+\boldsymbol{b}$ is still computed component-wise
- Special case: if $\boldsymbol{b}$ is the zero vector $\boldsymbol{0}$, then $\boldsymbol{a}+\boldsymbol{0}=\boldsymbol{a}$ — no displacement at all
- Collinearity is a degenerate case of the law, not a failure of it



CODE WALKTHROUGH · LA_01_03_02.IPYNB

Visualizing the Parallelogram Law

quiver() draws the arrows, plot() adds the helper edges

- (a)(b) Import NumPy and Matplotlib; define a_vec and b_vec
- (c) Compute a_plus_b = a_vec + b_vec
- (d) Create a square 6×6 canvas, figsize=(6,6)
- (e)(f) Draw a_vec in red and b_vec in blue with quiver()
- (g) Draw the pink resultant vector a_plus_b with quiver()
- (h)(i) Use plot() to add two dashed helper lines closing the parallelogram
- (j) Set equal aspect ratio, axis limits, grid lines, and legend

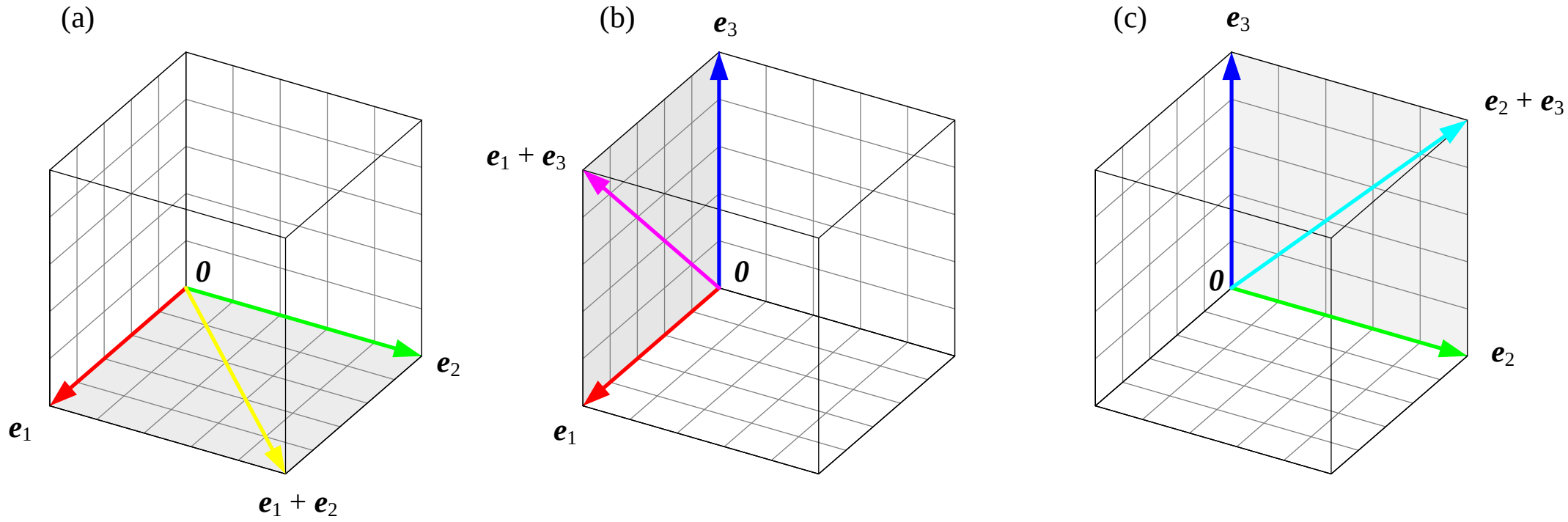
```python
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 定义向量
a_vec = np.array([4, 1])
b_vec = np.array([1, 2])

## 计算向量和
a_plus_b = a_vec + b_vec

## 可视化
plt.figure(figsize=(6,6))

# 绘制向量 a
plt.quiver(0, 0, a_vec[0], a_vec[1],
           angles='xy', scale_units='xy',
           scale=1, color='r', label="a")

# 绘制向量 b
plt.quiver(0, 0, b_vec[0], b_vec[1],
           angles='xy', scale_units='xy',
           scale=1, color='b', label="a")

# 绘制向量 a + b
plt.quiver(0, 0, a_plus_b[0], a_plus_b[1],
           angles='xy', scale_units='xy',
           scale=1, color='pink', label="a + b")

# 画出平行四边形的另外两条边
plt.plot([b_vec[0], a_plus_b[0]],
         [b_vec[1], a_plus_b[1]], 'k--')

plt.plot([a_vec[0], a_plus_b[0]],
         [a_vec[1], a_plus_b[1]], 'k--')

# 装饰
plt.gca().set_aspect('equal', adjustable='box')
plt.xlim(0, 5)
plt.ylim(0, 5)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(color='gray', alpha=0.8, linestyle='-', linewidth=0.25)
plt.legend()
```

Figure 5 · Mixing Two Primary Colors

- The parallelogram law also explains color mixing inside the 3-D RGB cube

- e_1+e_2=yellow, e_1+e_3=magenta, e_2+e_3=cyan
- Each secondary color sits on the diagonal of a parallelogram spanned by two base-color vectors

- The same 2-D geometric idea carries over directly into 3-D space



ADDITIVE IDENTITY

Zero Vector: $a+0=a$

$$\boldsymbol{a}+\boldsymbol{0}=\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}+\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}=\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

- Every component of the zero vector $\boldsymbol{0}$ is 0 — the additive identity for vector addition
- Adding the zero vector leaves any vector unchanged; geometrically, no displacement occurs

- In RGB terms, the zero vector $\boldsymbol{0}$ is black: no light added, still black
- Building up from black through $\boldsymbol{e_1}$, then $\boldsymbol{e_2}$, toward yellow shows addition "accumulating"

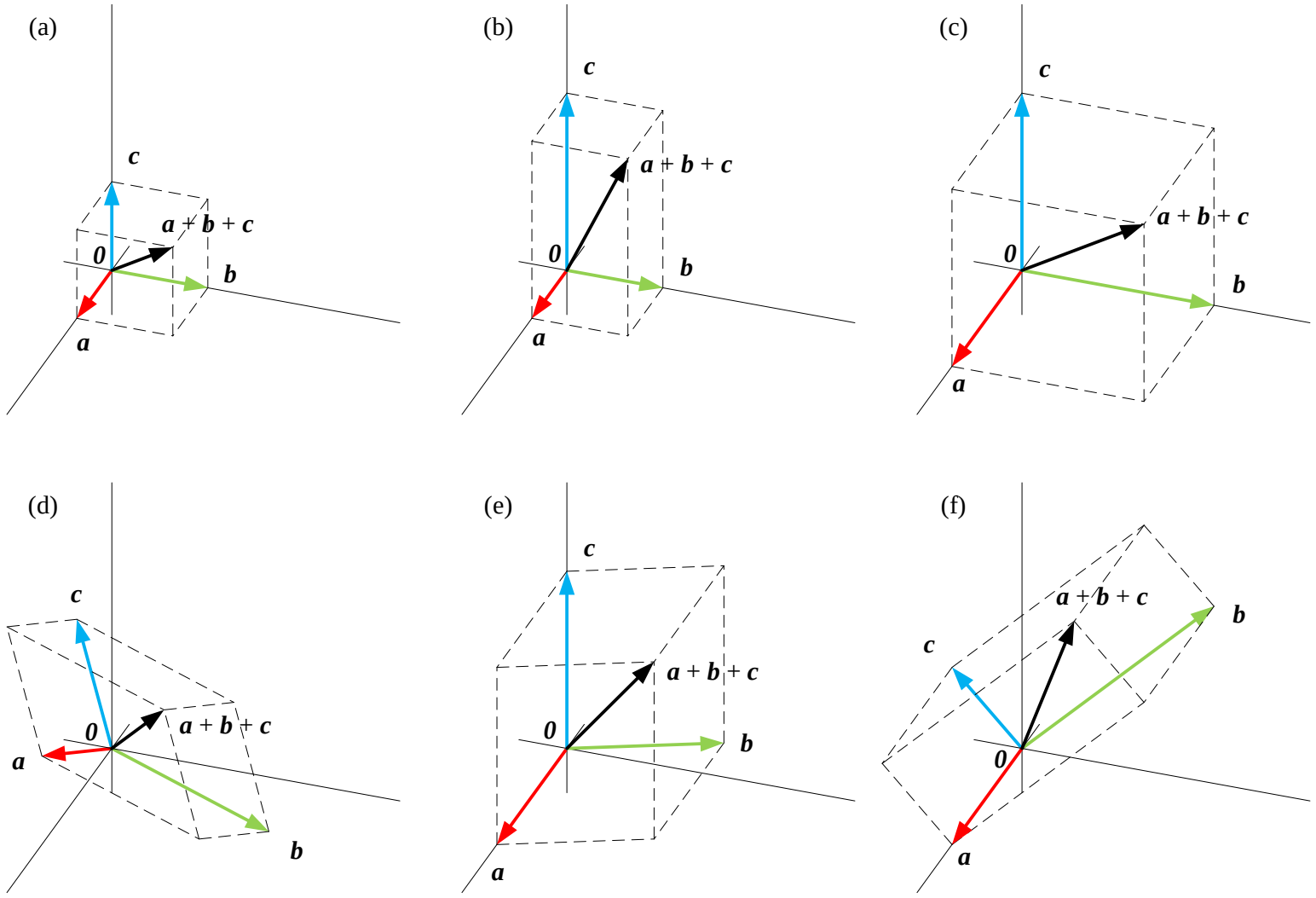
The Parallelepiped Rule

$\boldsymbol{a}$, $\boldsymbol{b}$, $\boldsymbol{c}$ non-coplanar → build a parallelepiped with these as edges → the space diagonal is $\boldsymbol{a}+\boldsymbol{b}+\boldsymbol{c}$

- Three non-coplanar vectors $\boldsymbol{a}$, $\boldsymbol{b}$, $\boldsymbol{c}$ share a start point and form the edges of a parallelepiped
- The space diagonal leaving that shared start point equals the sum $\boldsymbol{a}+\boldsymbol{b}+\boldsymbol{c}$
- A natural extension of the parallelogram law from 2-D into 3-D space

Figure 6 · Adding Three Non-Coplanar Vectors via the Parallelepiped

- (a)–(f) six different sets of ***a***, ***b***, ***c***, each shown against a cube grid
- Whatever the three directions, the space diagonal always equals their sum



SPECIAL CASE

Figure 7 · Three Coplanar Vectors

- When ***a***, ***b***, ***c*** are coplanar, no true parallelepiped can be formed
- (a) The figure degenerates to a 2-D parallelogram; the rule still holds

- (b) If the three vectors close up tip-to-tail into a triangle, then ***a+b+c=0***
- This closed-triangle result naturally leads into the triangle law below

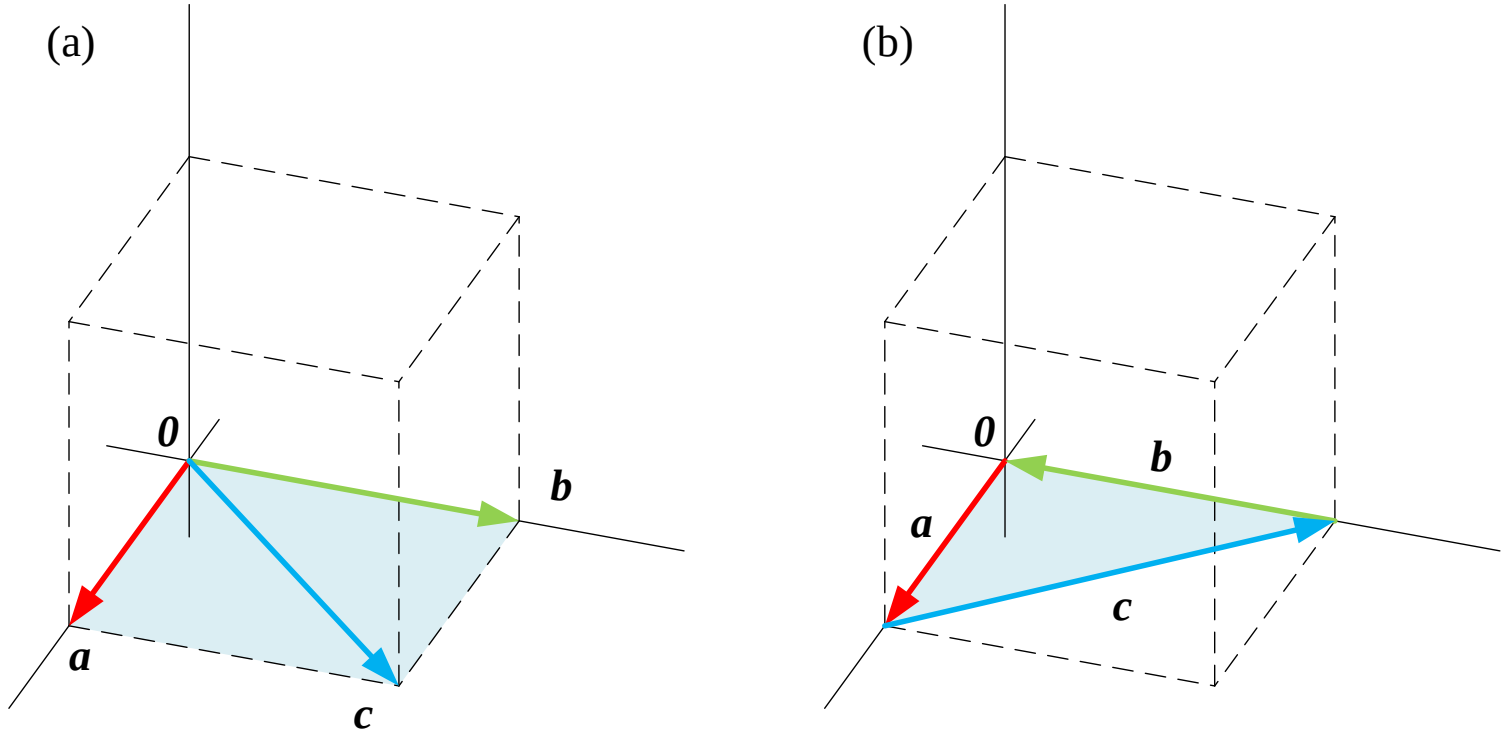


Figure 8 · The Unit Cube Computes $e_1+e_2+e_3$

- The three base-color vectors e_1, e_2, e_3 form the edges of the unit cube
- The tip of the space diagonal, $[1,1,1]^T$, is exactly where white sits

- Equal mixing of red, green, and blue yields white — geometry and color line up perfectly

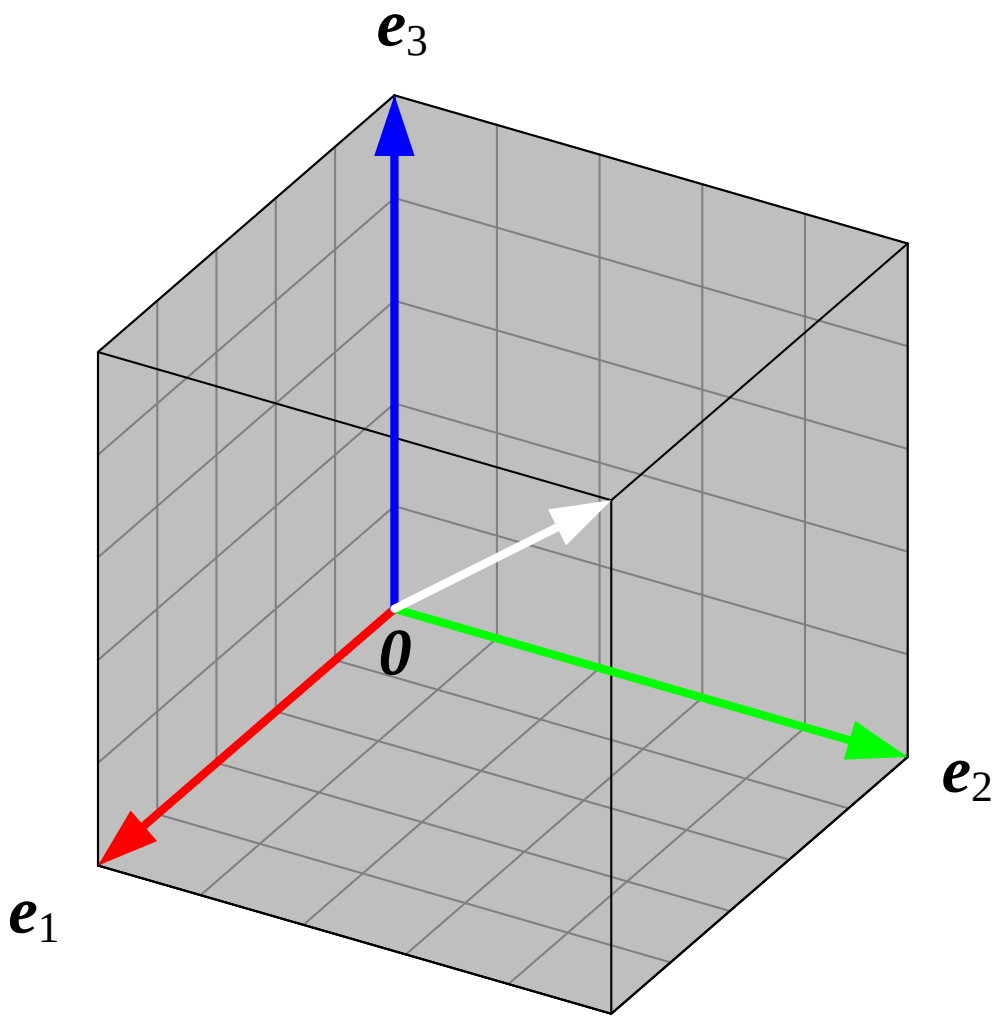
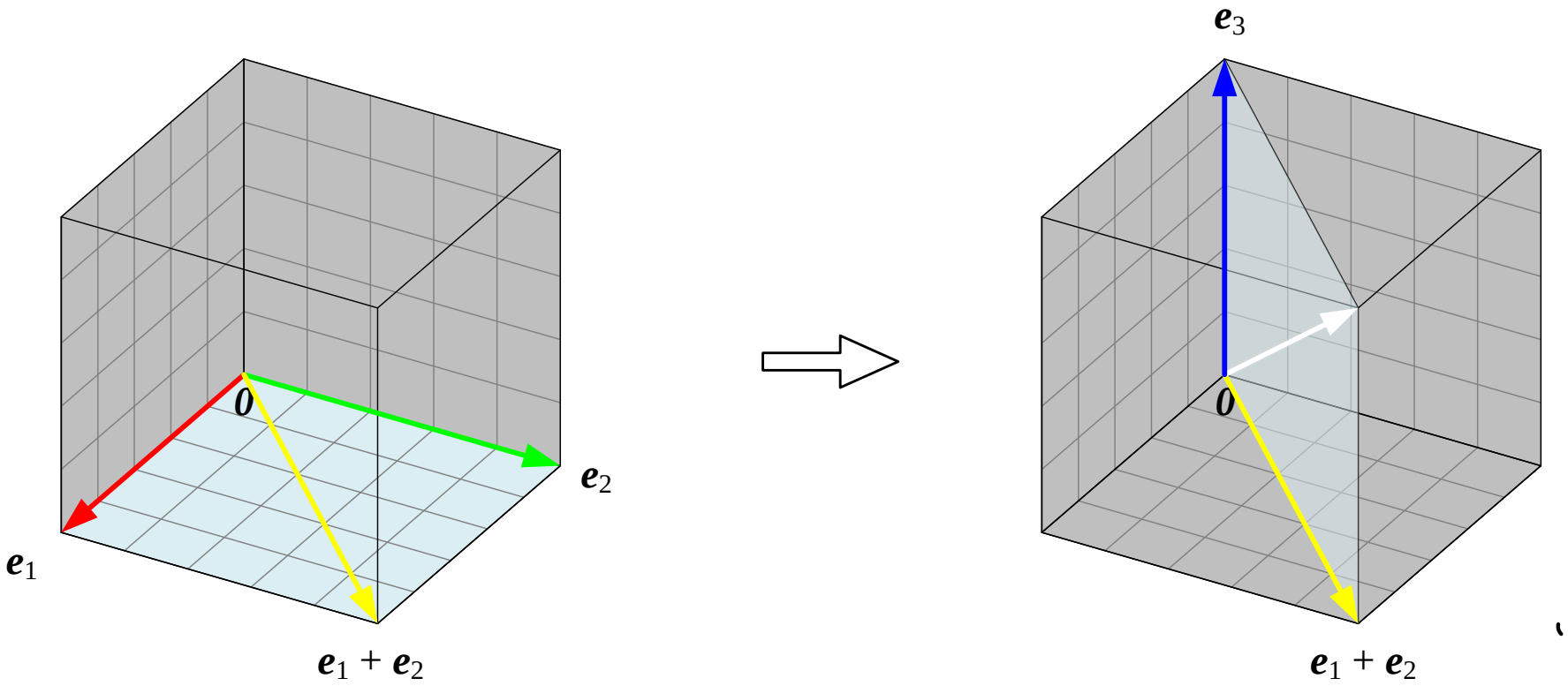


Figure 9 · Splitting Into Two Parallelograms: $(e_1+e_2)+e_3$

- The parallelepiped can also be built in two parallelogram steps
- Step 1: compute e_1+e_2 (yellow) — the base diagonal
- Step 2: add e_3 to that result to get the final space diagonal
- This decomposition foreshadows the associative law discussed later



GEOMETRIC VIEW

The Triangle Law

Translate $\mathbf{b}$ so its start point meets $\mathbf{a}$'s tip → the arrow from $\mathbf{a}$'s start to $\mathbf{b}$'s tip is $\mathbf{a}+\mathbf{b}$

- Slide ***b*** tip-to-tail so its start point coincides with the tip of ***a***
- The arrow from ***a***'s start point to ***b***'s tip is the resultant vector ***a***+***b***
- The triangle law and the parallelogram law are two ways of drawing the same result

Figure 10 · Examples of the Triangle Law for $a+b$

- (a)–(f) the same vector pairs as Figure 3, now solved with the triangle law
- Compare with Figure 3: both laws place the resultant vector at exactly the same point

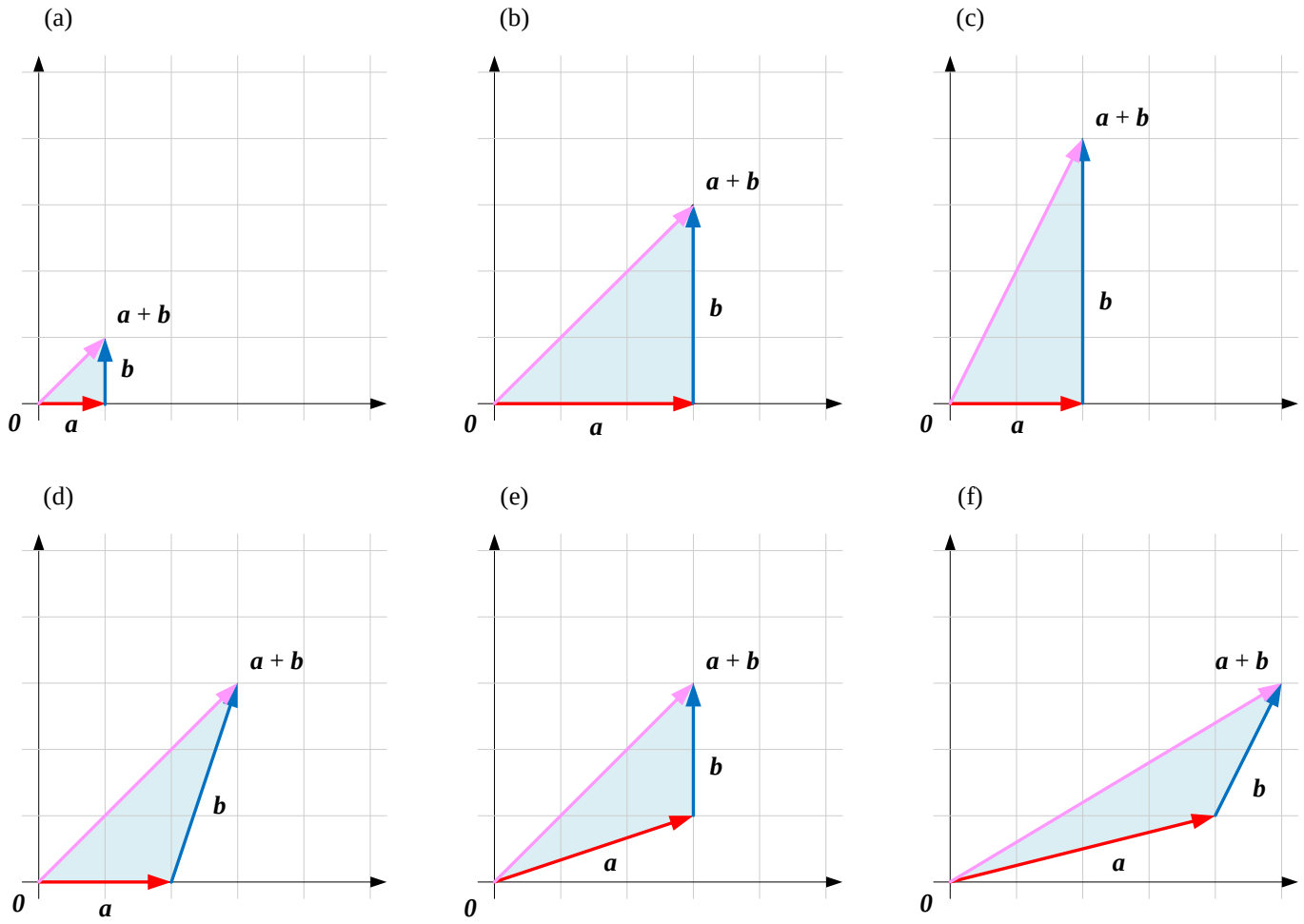
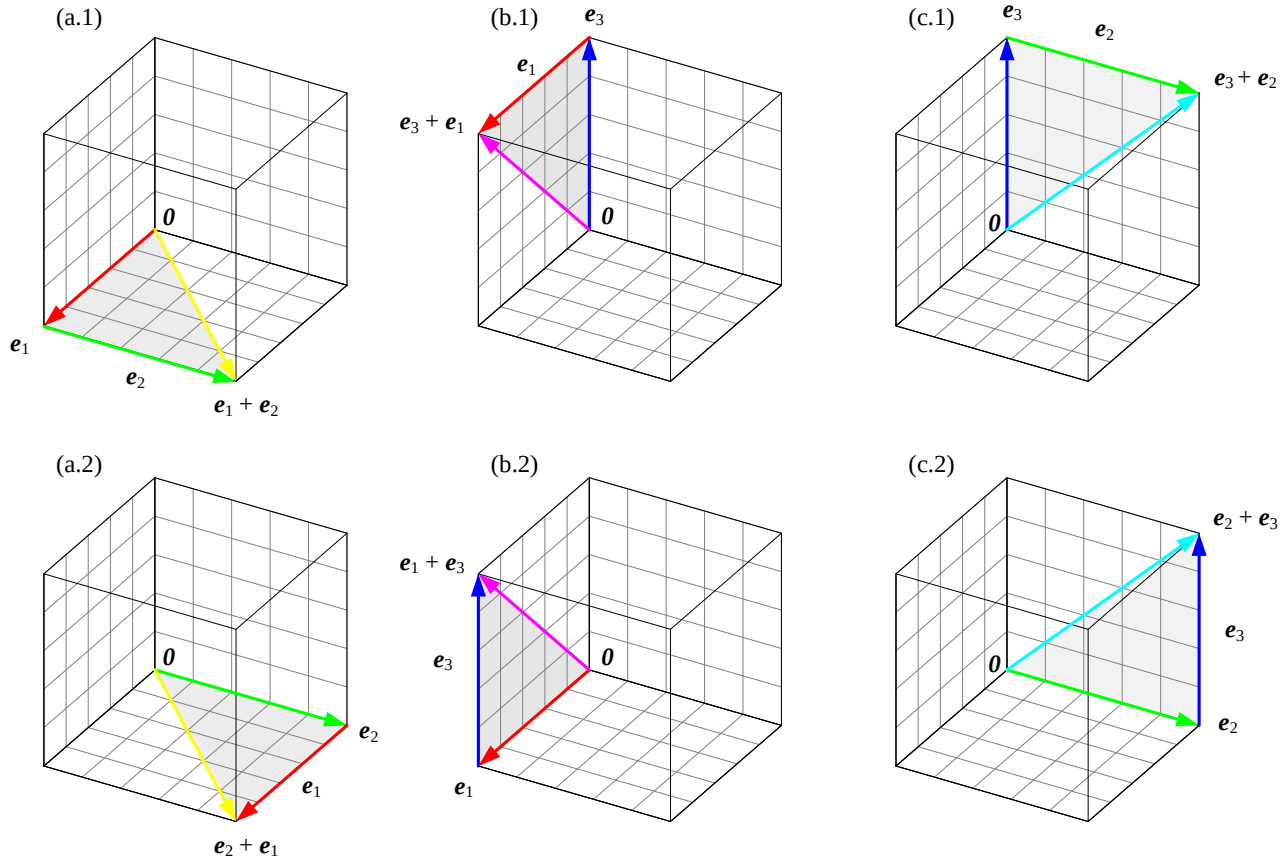


Figure 11 · Commutative Law: $e_1+e_2=e_2+e_1$

- a.1/a.2, b.1/b.2, c.1/c.2 show the two possible orders for each color pair
- Whichever vector is added first, the tip lands at the same point — and the same color



Adding Multiple Vectors: Tip-to-Tail Chaining

***a*+*b*+*c* = chain *a*, *b*, *c* tip-to-tail; the start-to-end arrow is the total sum**

- The triangle law extends directly to any number of vectors added in sequence
- Chain each vector tip-to-tail, then connect the first start point to the last tip
- That connecting arrow is the sum of all the vectors — no need to compute a parallelogram or parallelepiped first

Figure 12 · Adding Three Vectors Tip-to-Tail

- (a)–(f) six different sets of ***a***, ***b***, ***c***, chained tip-to-tail to reach the sum
- Compared with the parallelepiped in Figure 6, the triangle-law construction is more direct

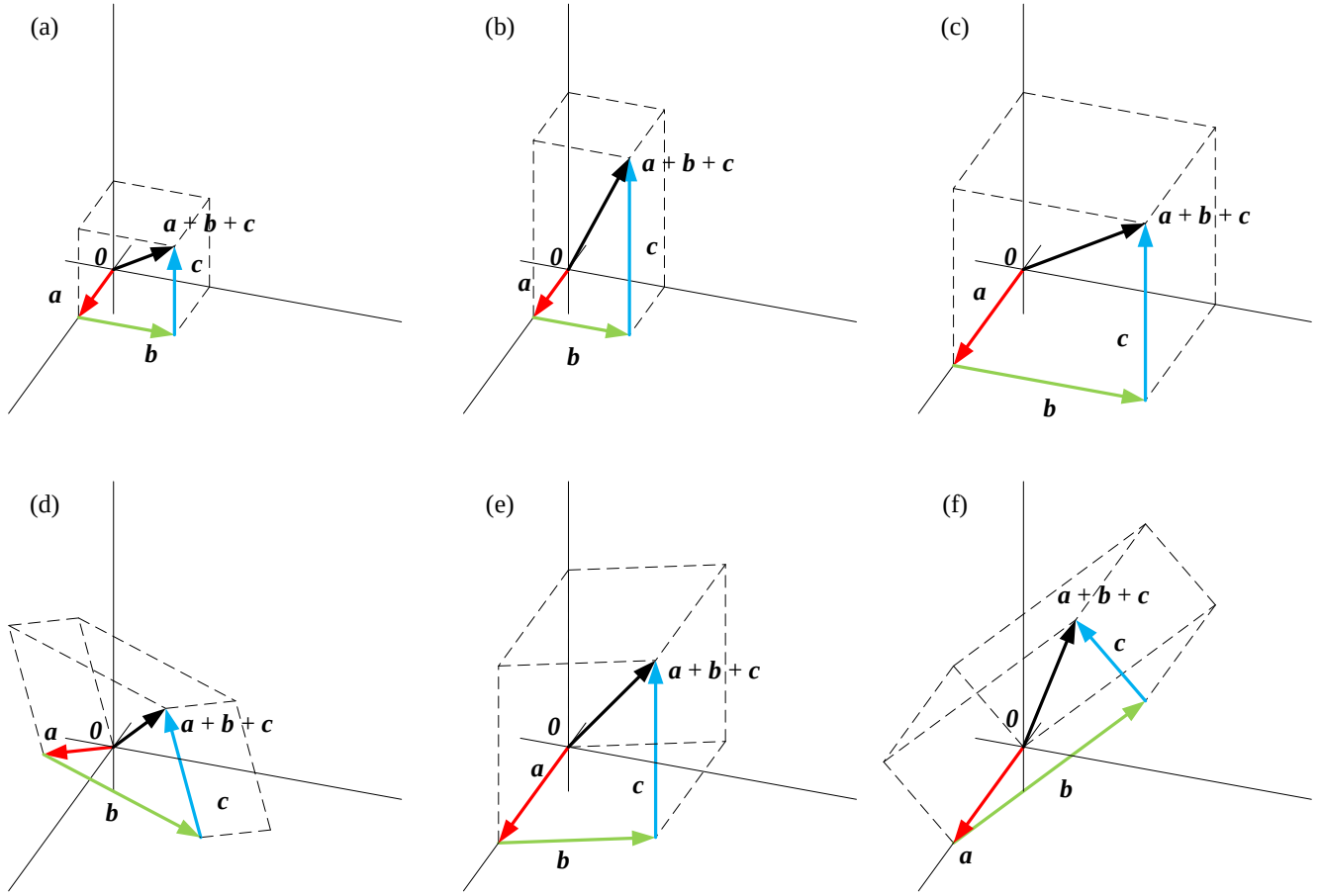
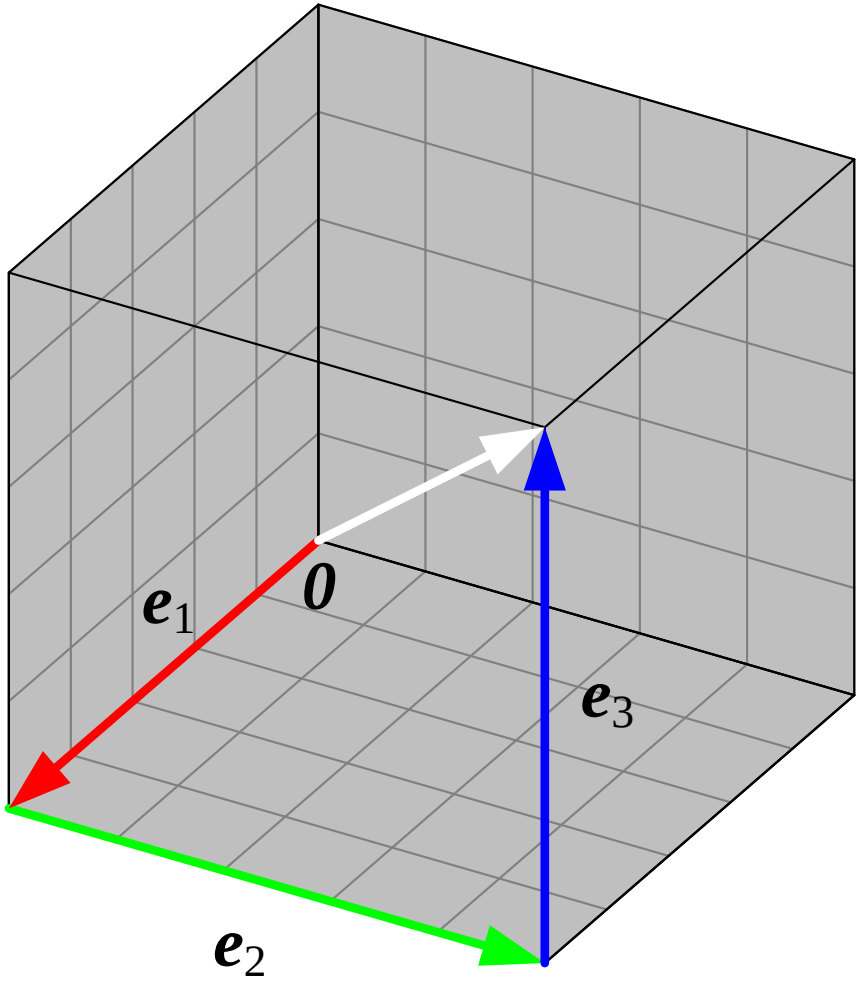


Figure 13 · Tip-to-Tail Sum $e_1+e_2+e_3$

- The red, green, and blue base-color vectors are chained tip-to-tail in turn
- The chain again ends at $[1,1,1]^T$ — the location of white
- This matches Figure 8's parallelepiped result, confirming the two pictures agree



The Associative Law

$$(e_1+e_2)+e_3 = e_1+(e_2+e_3)$$

- However the vectors are grouped, the final sum is identical — addition is associative

- Compute e_1+e_2 first and add e_3, or compute e_2+e_3 first and add e_1 — same result
- The associative law means the order of grouping never matters, only the tip-to-tail path itself

Figure 14 · Two Paths: $(e_1+e_2)+e_3$ and $e_1+(e_2+e_3)$

- Top row: compute e_3+e_2 first, then add e_1; bottom row: compute e_2+e_1 first, then add e_3
- Two completely different computation paths converge on the same white vertex

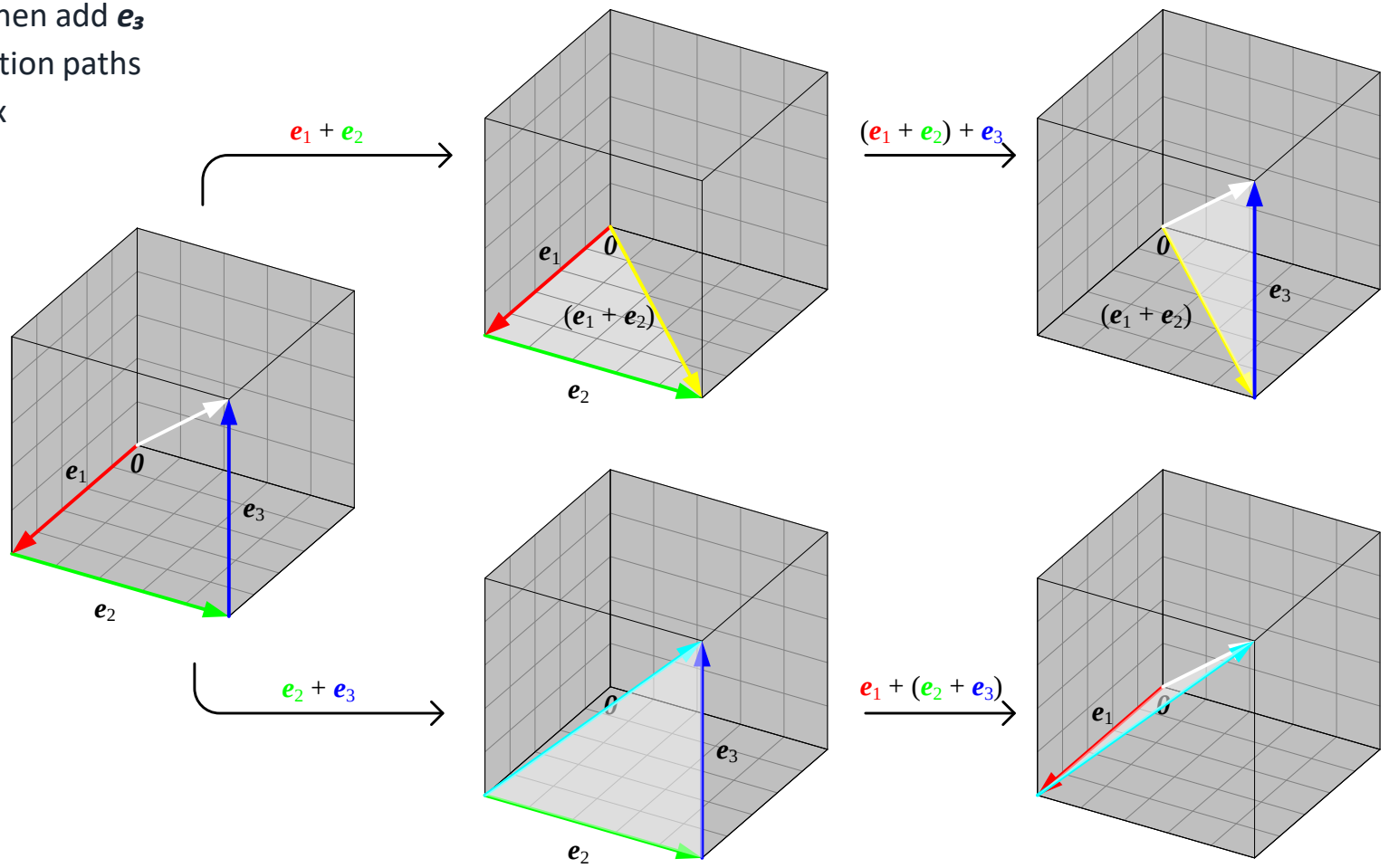
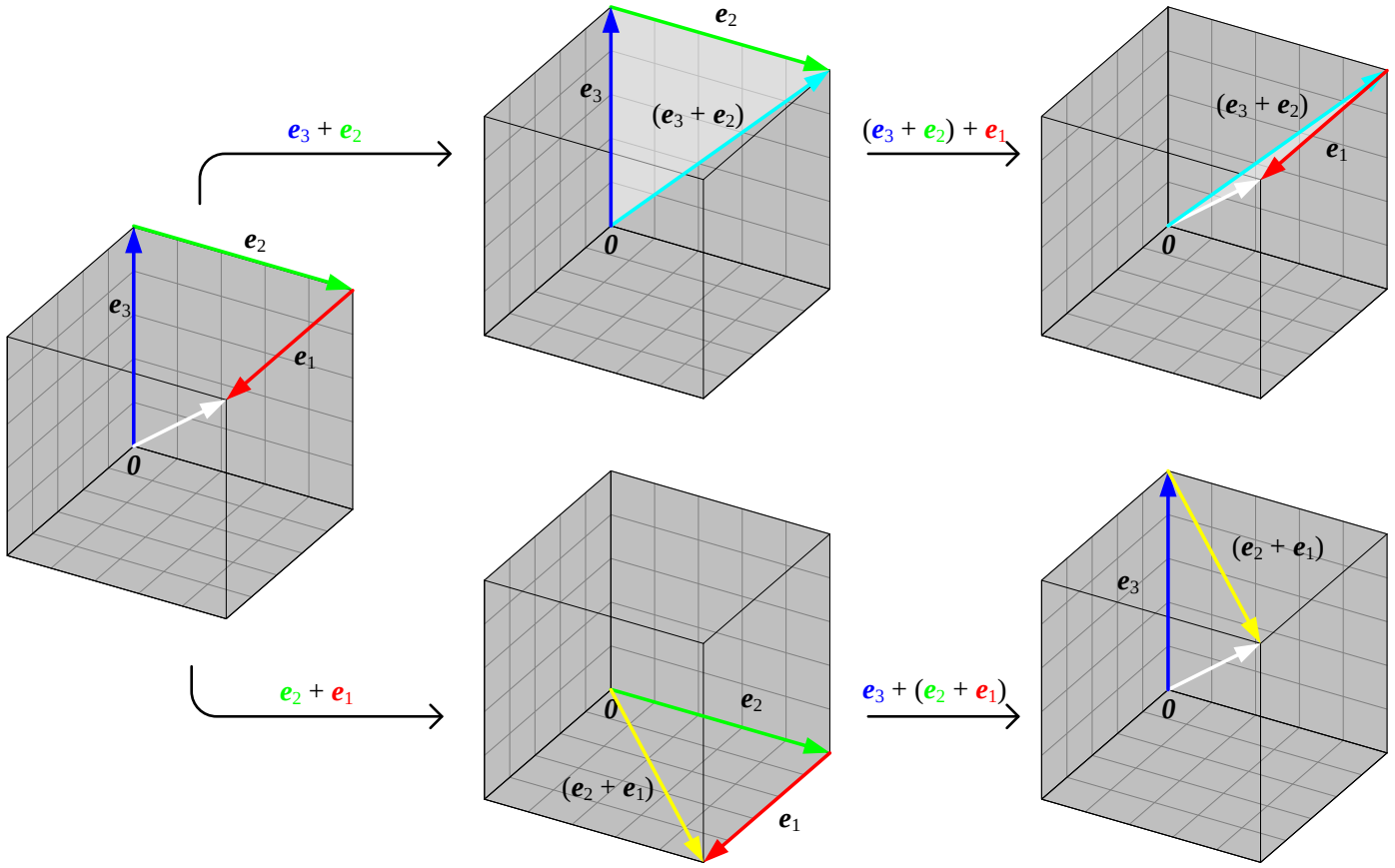


Figure 15 · Two Paths: $(e_3+e_2)+e_1$ and $e_3+(e_2+e_1)$

- Reordering the vectors and re-verifying the associative law still converges on white
- Think it over: what other orderings of red, green, and blue also reach the same white?



ALGEBRAIC DEFINITION

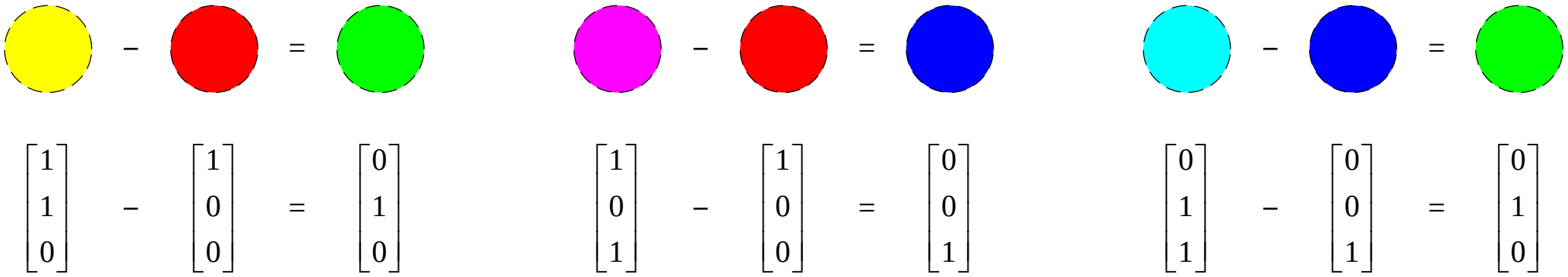
Vector Subtraction: Component by Component

$$\boldsymbol{a}-\boldsymbol{b} = [a_1;a_2;\cdots;a_n] - [b_1;b_2;\cdots;b_n] = [a_1-b_1;\ a_2-b_2;\ \cdots;\ a_n-b_n]$$

- Subtracting one n-D vector from another again requires matching dimensions, component by component
- In RGB terms, subtracting a color vector is like filtering that base color out of the mix
- Example: yellow (red+green) minus red leaves exactly the green component

Figure 16 · Vector Subtraction in RGB Color Space

- Yellow − red = green; magenta − red = blue; cyan − blue = green
- Color subtraction is component-wise subtraction — it visibly "filters out" a base color



$\boldsymbol{a-b = a+(-b)}$: the Opposite Vector

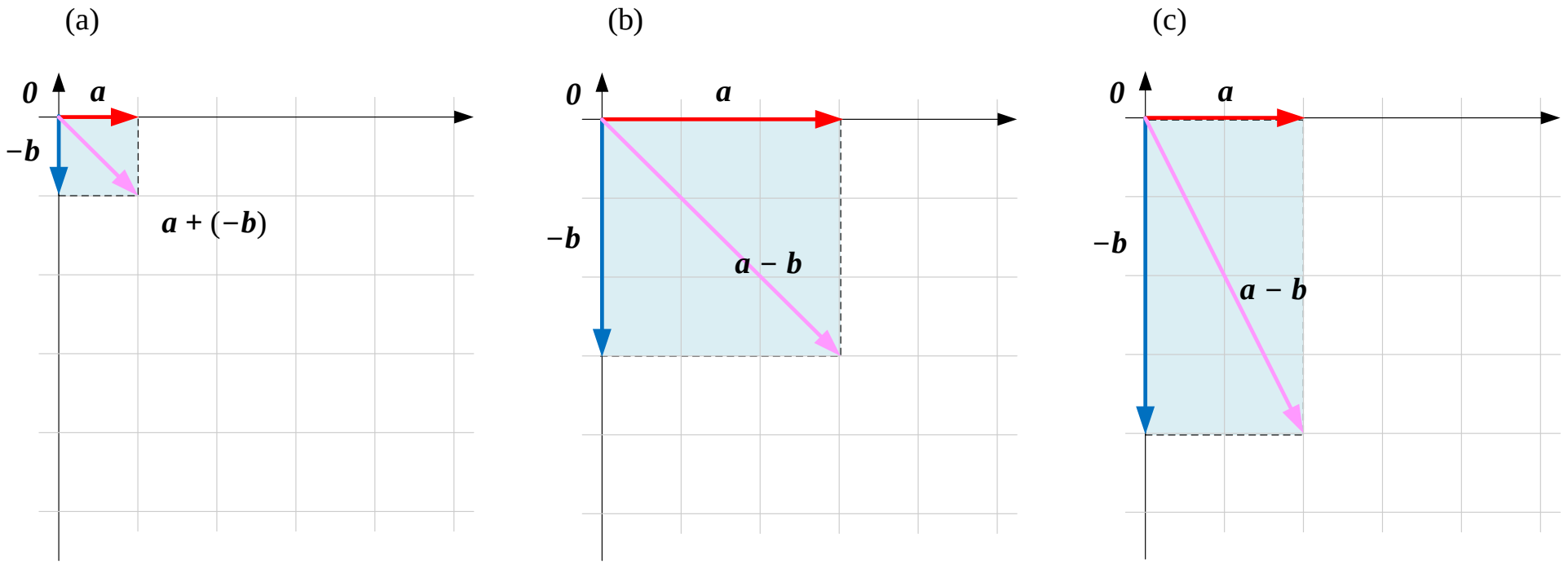
$\boldsymbol{a-b = a+(-b)}$, where $\boldsymbol{-b}$ is the opposite of $\boldsymbol{b}$ (same length, opposite direction)

- Vector subtraction can be read as: add the "opposite vector" of the vector being subtracted

- $\boldsymbol{-b}$ has the same length as $\boldsymbol{b}$ but points the opposite way, so $\boldsymbol{b}+(\boldsymbol{-b})=\boldsymbol{0}$
- Subtraction thereby reduces to familiar addition, so the parallelogram/triangle laws apply directly

Figure 17 · The Parallelogram Law for $a-b$

- First draw $-\boldsymbol{b}$, then add it to $\boldsymbol{a}$ with the parallelogram law to get $\boldsymbol{a}-\boldsymbol{b}$
- (a)–(c) three different pairs of $\boldsymbol{a}$, $\boldsymbol{b}$ show the geometric result of subtraction on the plane



GEOMETRIC DERIVATION

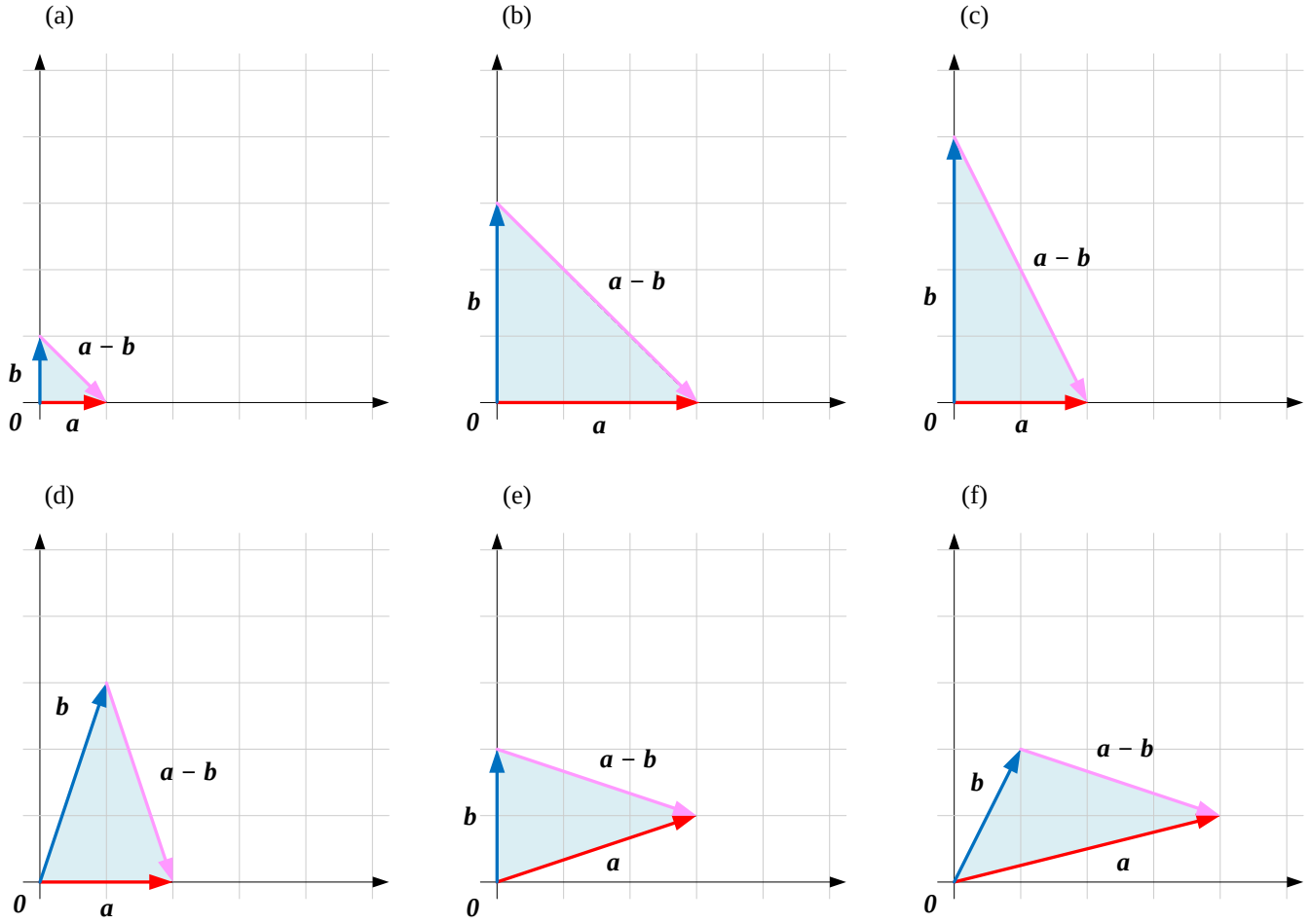
Let $c=a-b$, Then $b+c=a$

$$c = a-b \iff b+c = a \iff b+(a-b) = a$$

- If $\boldsymbol{c}$ is defined as $\boldsymbol{a}-\boldsymbol{b}$, then adding $\boldsymbol{b}$ and $\boldsymbol{c}$ must return $\boldsymbol{a}$
- This is the triangle law in reverse: the arrow from $\boldsymbol{b}$'s tip to $\boldsymbol{a}$'s tip is exactly $\boldsymbol{a}-\boldsymbol{b}$
- This gives a second geometric construction for subtraction, with no opposite vector required

Figure 18 · The Triangle Law for $a-b$

- (a)–(f) the arrow from $\boldsymbol{b}$'s tip to $\boldsymbol{a}$'s tip is the resultant vector $\boldsymbol{a}-\boldsymbol{b}$
- Compared with Figure 17's parallelogram law, the triangle-law construction is more direct



QUICK GLOSSARY

Key Terms in This Section

Vector Addition *逐分量相加 — component-wise sum:* $\boldsymbol{a+b}$

Vector Subtraction *逐分量相减 — component-wise difference:* $\boldsymbol{a-b}$

Parallelogram Law *共起点补全平行四边形，对角线为和*

Triangle Law *首尾相连，起点到终点为和*

Parallelepiped Rule *三个不共面向量相加的空间图像*

Commutative Law $\boldsymbol{a+b = b+a}$

Associative Law $\boldsymbol{(a+b)+c = a+(b+c)}$

Zero Vector $\boldsymbol{a+0 = a}$，*additive identity*

Opposite Vector $\boldsymbol{-b}$，*satisfies* $\boldsymbol{b+(-b)=0}$

Collinear / Coplanar *degenerate case of the parallelogram / parallelepiped*

FURTHER LEARNING

Companion Code & Extended Reading

Code 1 · LA_01_03_01.ipynb
Basic vector addition and subtraction with NumPy

Code 2 · LA_01_03_02.ipynb
Visualizing the parallelogram law with Matplotlib

Code 3 · LA_01_03_03.ipynb (self-study)
Visualizing the triangle law, reinforcing the tip-to-tail construction

Open-Source Repository
Full code and figure assets: https://github.com/Visualize-ML

Q1 – Q10

Q1 Use np.array() to create $\boldsymbol{a}=[1,2,3]^T$, $\boldsymbol{b}=[3,4,5]^T$, $\boldsymbol{c}=[5,6,7]^T$, and compute $\boldsymbol{a}+\boldsymbol{b}+\boldsymbol{c}$.

Q2 Use Code 2 to visualize each panel (a)–(f) of Figure 3.

Q3 Learn matplotlib.pyplot.fill() and modify Code 2 to shade the parallelogram.

Q4 Modify Code 1 to compute $\boldsymbol{a}-\boldsymbol{b}$ and $\boldsymbol{b}-\boldsymbol{a}$ for $\boldsymbol{a}=[3,2,1]^T$, $\boldsymbol{b}=[5,4,3]^T$.

Q5 Write Python code to reproduce Figure 8 (the RGB parallelepiped, $\boldsymbol{e}_1+\boldsymbol{e}_2+\boldsymbol{e}_3$).

Q6 Write Python code to reproduce Figure 13 (the RGB triangle law, $\boldsymbol{e}_1+\boldsymbol{e}_2+\boldsymbol{e}_3$=white).

Q7 Use the parallelogram law to visualize $\boldsymbol{a}-\boldsymbol{b}$ for panels (d)(e)(f) of Figure 3.

Q8 Based on Figure 18, use Python to plot $\boldsymbol{b}-\boldsymbol{a}$.

Q9 Randomly generate two 3-D vectors with NumPy and verify $\boldsymbol{a}+\boldsymbol{b}=\boldsymbol{b}+\boldsymbol{a}$.

Q10 Randomly generate three 3-D vectors with NumPy and verify $(\boldsymbol{a}+\boldsymbol{b})+\boldsymbol{c}=\boldsymbol{a}+(\boldsymbol{b}+\boldsymbol{c})$.

Q11 – Q19

Q11 Write parallelogram(a, b) to draw the parallelogram law for two 2-D vectors with Matplotlib.

Q12 Write triangle(a, b) to draw the triangle law for two 2-D vectors with Matplotlib.

Q13 Use quiver() to plot $\boldsymbol{a}$, $\boldsymbol{b}$, $\boldsymbol{a}+\boldsymbol{b}$, and $\boldsymbol{a}-\boldsymbol{b}$ together and compare directions.

Q14 Generate 100 random pairs of 2-D vectors and report the fraction that are collinear (degenerate).

Q15 Implement the 3-D parallelepiped rule with NumPy and verify the vertex equals the vector sum.

Q16 Write code to test whether three vectors are coplanar (hint: scalar triple product / determinant = 0).

Q17 Use Matplotlib's 3-D toolkit to draw the RGB cube and mark the $\boldsymbol{e}_1+\boldsymbol{e}_2+\boldsymbol{e}_3$ diagonal.

Q18 Write vector_chain(vectors) to visualize the tip-to-tail sum of any number of vectors.

Q19 Verify in Python whether $(\boldsymbol{e}_3+\boldsymbol{e}_2)+\boldsymbol{e}_1$ equals $\boldsymbol{e}_3+(\boldsymbol{e}_2+\boldsymbol{e}_1)$, and consider what other orderings work.

Q20 – Q28

Q20 Implement $\boldsymbol{a}-\boldsymbol{b}$ with NumPy and verify it equals $\boldsymbol{a}+(-\boldsymbol{b})$.

Q21 Given vectors $\boldsymbol{a}$ and $\boldsymbol{c}$, write code to solve for $\boldsymbol{b}$ satisfying $\boldsymbol{b}+\boldsymbol{c}=\boldsymbol{a}$.

Q22 Implement component-wise addition with an explicit for-loop and compare it against NumPy's result.

Q23 Write is_zero_vector(v) to test whether a vector is the zero vector, allowing floating-point tolerance.

Q24 Generate a random n-D vector $\boldsymbol{a}$ in Python and verify $\boldsymbol{a}+\boldsymbol{0}=\boldsymbol{a}$.

Q25 Use a Matplotlib animation (FuncAnimation) to show $\boldsymbol{a}+\boldsymbol{b}$ building up from 0 to its final tip.

Q26 For every 0/1 combination of the three RGB base-color vectors, list all resulting mixed colors.

Q27 Use NumPy to verify that in n-D space, any vector can be written as a weighted sum of n basis vectors.

Q28 Write subtract_and_plot(a, b) to draw $\boldsymbol{a}-\boldsymbol{b}$ with both the parallelogram and triangle laws, and compare.

TALK TO AI

10 Prompts to Explore Further (1/2)

1. Using an everyday example, explain why vector addition uses the "parallelogram law" instead of simply adding the two arrows' lengths.

2. Prove that 2-D and 3-D vector addition satisfies the commutative and associative laws, and discuss whether this still holds in higher dimensions.

3. Explain why three coplanar vectors cannot form a true parallelepiped, with a diagram to support your explanation.

4. If additive RGB color mixing is viewed as vector addition, what real-world phenomenon corresponds to vector subtraction?

5. Design a small Python experiment that checks whether the sum of several vectors stays the same under any order of addition.

TALK TO AI

10 Prompts to Explore Further (2/2)

6 What do a vector's "opposite vector" $-\boldsymbol{b}$ and a scalar's "additive inverse" −k have in common, and how do they differ?

7 Explain why the parallelogram law and the triangle law are really the same idea drawn two different ways, and when each is more convenient.

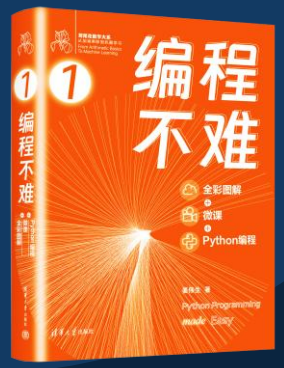
8 When two vectors are collinear, the parallelogram law degenerates — describe the resulting figure and give an algebraic explanation.

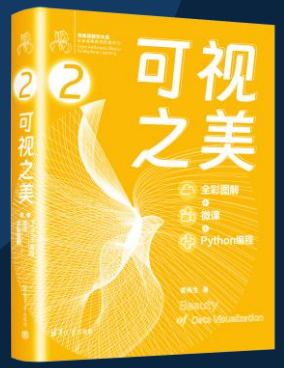
9 Reflect on how the "zero vector" in vector addition is conceptually similar to the "zero matrix" in matrix addition.

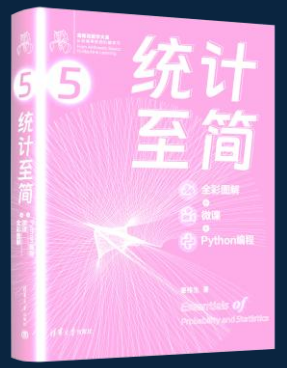
10 Use a concrete physical scenario (such as forces, displacements, or velocities) to explain the real-world meaning of vector addition and subtraction.

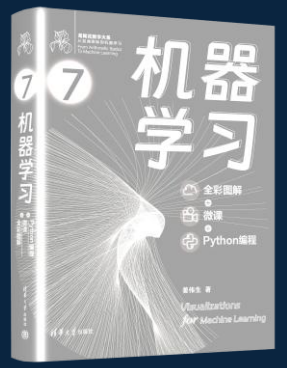
数学

数学基础　　线性代数　　概率统计

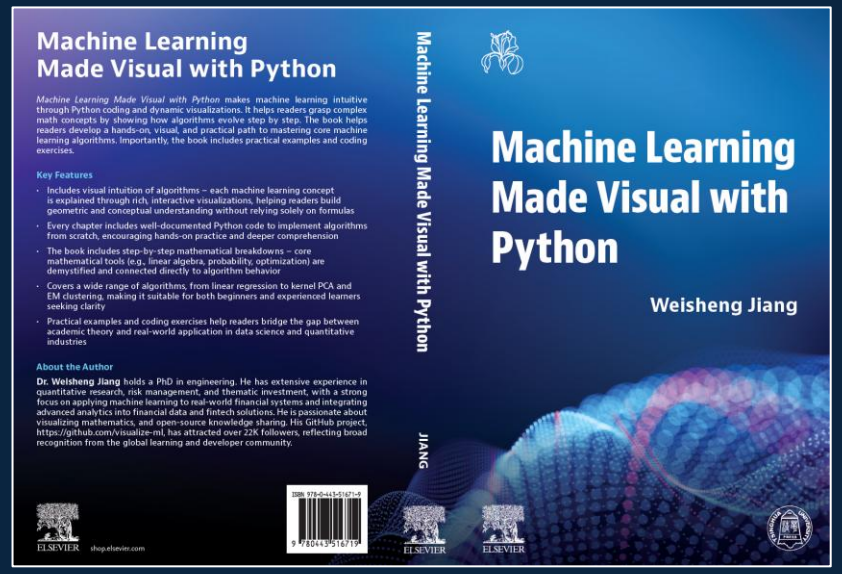








Python编程　　数据可视化



数据分析
图和网络　　回归、降维
分类、聚类

工具　　实践